

1. Introduction and Objective

In small electronics and medical laboratories, sensitive equipment is damaged by sudden temperature swings and excessive humidity. This mini-project implements an IoT embedded system on a **Raspberry Pi** that (i) measures temperature and humidity with a **DHT11** sensor every 15 s, (ii) uploads each reading to the **ThingSpeak** cloud for storage and live graphs, (iii) trains a **Linear Regression** model to predict the temperature trend, and (iv) automatically switches a **fan** (cooling) and a **heater** (heating) to keep the temperature inside the band 24 °C to 26 °C. In this prototype the fan and heater are represented by two indicator LEDs.

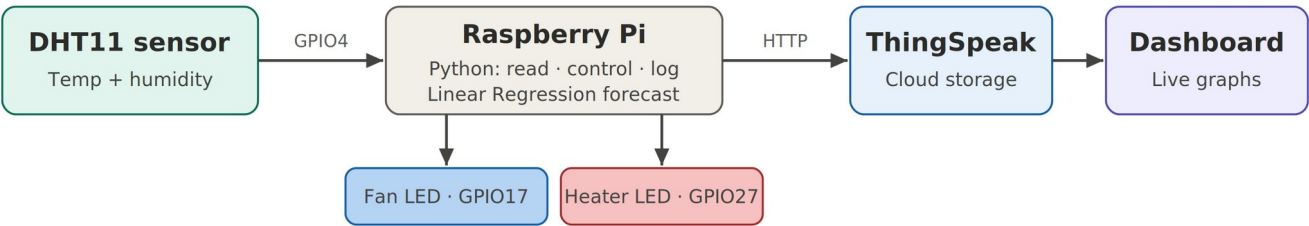


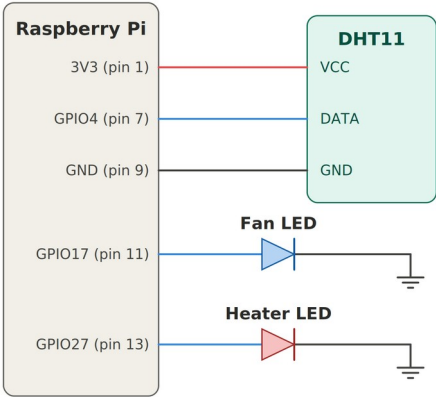
Figure 1: System block diagram

2. Hardware Design

- Raspberry Pi running Raspberry Pi OS with Python 3 (programmed in Thonny)
- DHT11 temperature and humidity sensor (accuracy about ±2 °C and ±5 % RH)
- Two LEDs (fan and heater), each with a 330 Ω series resistor
- Breadboard and jumper wires

Pins use BCM (Broadcom GPIO) numbering, with the physical header pin in brackets.

Component pin	Raspberry Pi pin	Function
DHT11 VCC	3V3 (pin 1)	Sensor power
DHT11 DATA	GPIO4 (pin 7)	Data (10 kΩ pull-up to 3V3 on a bare sensor)
DHT11 GND	GND (pin 9)	Ground
Fan LED anode	GPIO17 (pin 11)	Cooling indicator
Heater LED anode	GPIO27 (pin 13)	Heating indicator
LED cathodes	GND	Return path



IoT temperature monitor wiring (BCM numbering)

Figure 2: Circuit schematic (simplified). In practice each LED has a 330 Ω series resistor.

3. Software Design and Control Logic

A single Python script uses **adafruit_dht** (sensor), **gpiozero** (LEDs), **numpy** (regression), **matplotlib** (graph) and **requests** (cloud). Every 15 s it performs this cycle:

1. **Read** temperature and humidity (up to 5 retries, because the DHT11 occasionally misses a read).
2. **Control** the fan and heater LEDs by comparing the temperature with the thresholds (table).
3. **Log** the reading with a timestamp to readings.csv.
4. **Predict**: every 10 readings, fit the regression line, print its equation and save trend.png.
5. **Upload** temperature, humidity, fan state, heater state and the prediction to ThingSpeak.

Temperature T	Fan	Heater
$T \geq 26\text{ °C}$ (MAX_TEMP)	ON	OFF
$T \leq 24\text{ °C}$ (MIN_TEMP)	OFF	ON
$24\text{ °C} < T < 26\text{ °C}$	OFF	OFF

Background operation. The script runs as a systemd service that starts at boot:

```
[Unit] Description=IoT Temp Monitor
After=network-online.target
[Service] User=ett WorkingDirectory=/home/ett
ExecStart=/usr/bin/python3 /home/ett/asss.py
Restart=always
[Install] WantedBy=multi-user.target
$ sudo systemctl enable --now monitor.service
```

4. Cloud Storage and Dashboard

Data is sent to a ThingSpeak channel with an HTTP GET request to its update API on every cycle (15 s matches the minimum update interval of the free plan). The channel has five fields: **Field 1** Temperature, **Field 2** Humidity, **Field 3** Fan state (1 = ON), **Field 4** Heater state (1 = ON) and **Field 5** Predicted temperature. ThingSpeak stores every entry, so the history can be exported for later analysis, and plots each field as a live graph that serves as the monitoring dashboard.

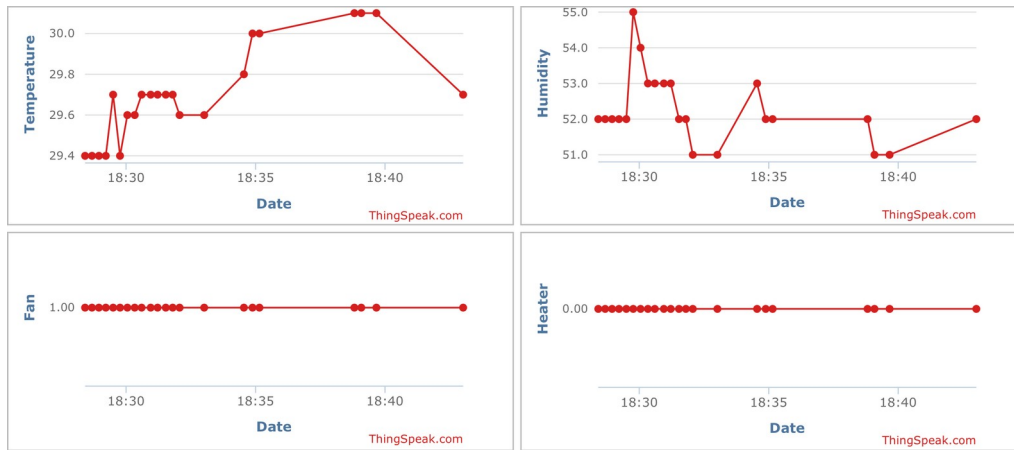


Figure 3: ThingSpeak dashboard: Temperature, Humidity, Fan state and Heater state (log of 2 October 2026, about 18:28 to 18:43)

5. Machine Learning: Linear Regression

The model fits a straight line $T = m \cdot t + c$ to the logged readings, where t is the time in minutes since the first reading. The slope m and intercept c come from least squares (`numpy.polyfit`), and the fit quality is measured by $R^2 = 1 - SS_{res} / SS_{tot}$. The line is extended 30 minutes ahead to forecast the temperature, so heating or cooling can be anticipated. The forecast is also uploaded to Field 5.

Console output of one run:

```
[AI model] T = 0.0201*t + 29.66 (R^2=0.430)
          predicted in 30 min: 30.80C
```

The coefficients change as data accumulates. The saved graph (Figure 4) shows $T = 0.0629t + 29.44$ early in the run. Fitting all 94 logged readings gives $T = 0.0314t + 29.54$ with $R^2 = 0.52$, a rise of about 1.9 °C per hour and a forecast of about 31.5 °C 30 min after the last reading.

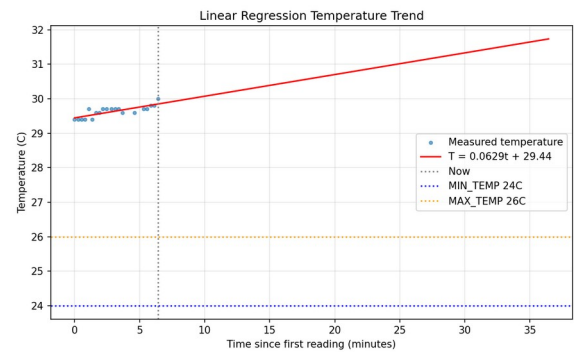


Figure 4: Regression line and equation (trend.png)



Figure 5: Predicted temperature (Field 5)

6. Results and Discussion

Measure (94 readings, 33 min)	Result
Temperature min / mean / max	29.4 / 30.0 / 31.6 °C
Humidity min / mean / max	50 / 51.8 / 56 %
Readings with $T \geq 26$ °C	100 % (fan ON)
Readings with $T \leq 24$ °C	0 % (heater OFF)

- The room stayed above MAX_TEMP for the whole run, so the fan LED stayed ON and the heater LED OFF, matching Fields 3 and 4. The heater branch can be tested with SIMULATE=1, which generates random readings from 20 °C to 32 °C.
- The cycle median is 16 s. A few longer gaps (up to 164 s) occurred when the DHT11 failed to read and the script retrieved.
- Limitations:** the DHT11 is coarse, the 2 °C band has no hysteresis (LEDs could flicker near a threshold), a straight line cannot capture non-linear changes (R^2 of 0.43 to 0.52), and a real fan or heater needs a relay or MOSFET driver with its own supply.

7. Conclusion

The prototype meets the project objectives: it measures temperature and humidity every 15 s, stores and plots the data on ThingSpeak, predicts the temperature trend with Linear Regression (equation and graph produced), controls a fan and heater automatically and runs continuously as a boot-time service. Future work includes a relay-driven fan and heater, hysteresis in the control logic, humidity control, a rolling-window regression, and a more accurate sensor such as the DHT22 or BME280.

Source code and project files: <https://github.com/ltzspencer235/iot-monitoring-project>